

Question 1:

In the function example2

```
for (int j=0; j < n; j += 2)
```

This loop increases by 2 each time, so it runs about $n/2$ times.

And because each iteration does constant-time operation which is $(total += arr[j])$, the running time is represented $O(n)$.

In the function example3

```
for (int j=0; j < n; j++)
```

```
    for (int k=0; k <= j; k++)
```

The loop in the outer part will run n amount of times.

The inner loop will run:

1 time when $j = 0$

2 times when $j = 1$

3 times when $j = 2$

...

n times when $j = n - 1$

This can be looked at in the expression:

$1 + 2 + 3 + \dots + n$

If we use the arithmetic formula:

$n(n+1)/2$

This simplifies to $O(n^2)$

Question 2:

```
import java.util.Random;

class Question2{
    public static void main(String[] args) {
        int n = 10;
        int[] A = new int[n-1];
        int rand = new Random().nextInt(n);
        for(int x = 0; x < n-1; x++){
            if (x < rand){
                A[x] = x;
            }else{
                A[x] = x+1;
            }
        }

        System.out.print("Array A >>> ");
        for(int x = 0; x < A.length; x++){
            System.out.print(A[x]);
            if (A.length - x > 1){
                System.out.print(", ");
            }
        }
        System.out.println(" (missing number " + rand + ")");

        System.out.println("Algorithm detects number " +
find_Missing_Number(A) + " is missing.");
    }

    public static int find_Missing_Number(int[] A){
        int array_Sum = 0;
        int natural_Sum = 0;
        for(int x = 0; x < A.length; x++){
            natural_Sum += x+1;
            array_Sum += A[x];
        }
        return natural_Sum - array_Sum;
    }
}
```

Output:

Ex 1: Array A >>> 0, 1, 2, 3, 4, 5, 6, 7, 9 (missing number 8)

Algorithm detects number 8 is missing.

Ex 2: Array A >>> 0, 1, 2, 3, 5, 6, 7, 8, 9 (missing number 4)

Algorithm detects number 4 is missing.

Question 3:

The best case running time is when the array is sorted in ascending order

Example:

[1,2,3,4,5]

What happens is for each iteration of the loop which is:

while (j >= 0 && arr[j] > key)

the condition $\text{arr}[j] > \text{key}$ is immediately false because every element is already in the correct position

So the while loop will execute 0 times for each pass and only 1 comparison is made for each iteration.

The outer loop will run $n - 1$ times, so it will grow linearly with n

The worst case running time is when the array is sorted in descending order

Each element (which is key) is compared with all previously sorted elements and shifted to the beginning of the array

Example:

2nd element compares once

3rd element compared twice

4th element compares 3 times

...

n th element compares $n-1$ times

This operation can be expressed by $1+2+3+\dots+(n-1)$

This turns into the equation:

$$n(n-1)/2$$

Which grows to:

$$O(n^2)$$

In short:

Best case is $O(n)$ where the array is already sorted

Worst case is $O(n^2)$ where the array is in reverse order